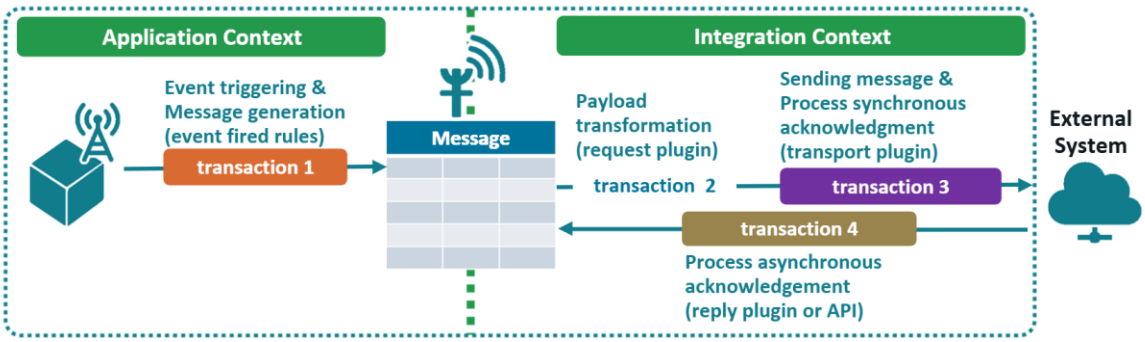
Sending Messages

Guidewire servers and messaging transactions



- All activity from **sending messages** takes place in **transaction 3**
- Messaging occurs across multiple transactions
  - If an exception is thrown, work in the current transaction is rolled back

Notes

Guidewire servers and messaging transactions

The servers involved at each point of the process are important because they determine where payload work should occur. The majority of payload generation should be done in Event Fired rules on the **ui** role server that manages the given user's session. However, if some portion of payload generation must be done immediately before the message is sent, that portion of the payload can be generated on the **messaging** role server.

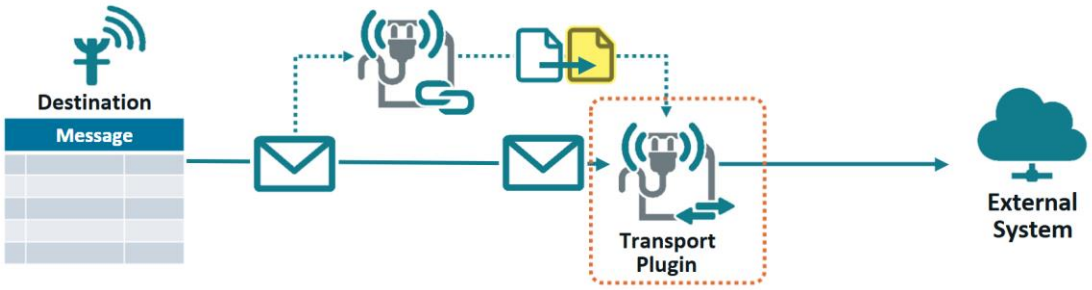
A message is always processed in at least two and as many as four transactions:

- Transaction 1 – Creates the message and generates the payload. This transaction always occurs.
- Transaction 2 – Prior to sending the message, an optional transaction may transform the message payload. This is usually done when the message must include information that can only be known immediately before sending the message.
- Transaction 3 – The sending of the message to the external system occurs in its own transaction. This transaction always occurs. If the external system acknowledges the message synchronously, then the acknowledgement is included in this transaction.
- Transaction 4 – If the external system acknowledges the message asynchronously, then a separate transaction processes the acknowledgement. This transaction does not occur for messages that are acknowledged synchronously.

The transaction structure is important because this determines the work that is rolled back if an error occurs.

Transport plugin

Message transport plugin



- The message transport plugin is responsible for sending the message
- The message transport plugin may also become responsible for:
  - Persisting the SenderRefID in a message if the SenderRefID is provided by an external system
  - Acknowledging in case of synchronous message exchange with an external system

Notes

Message transport plugin

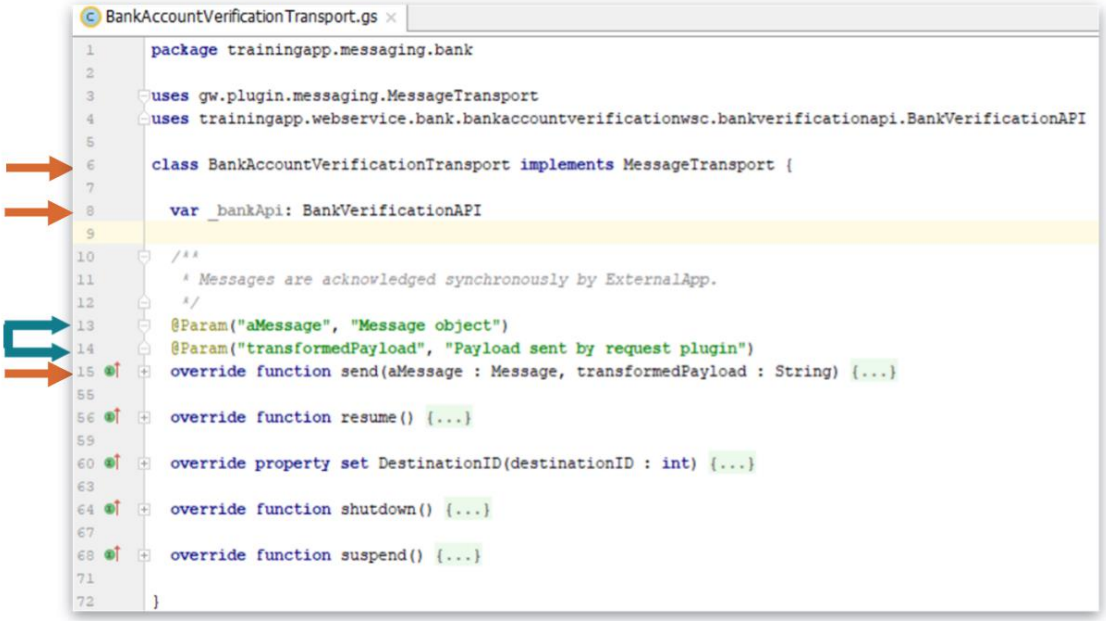
Communication with the external system may use *any* type of connectivity or transport, including some not normally considered messaging, such as:

- File-based, shared database
- Raw socket (TCP/IP)
- FTP
- Email
- Synchronous invocations, such as RMI/JRMP, RMI/IIOP, and SOAP.
- Asynchronous, message-based transports, such as JMS, native WebSphere MQ, or SOAP/JMS.

A message transport plugin can optionally implement the InitializablePlugin interface, which adds the set Parameters property to the plugin.

Create a message transport plugin class

- Use a package naming convention recommendation:
  - **<company>.<app_code>.messaging.<functional_area>**



Notes

Create a message transport plugin class

Code example

Line 6
- Implement the MessageTransport interface.
- Message entity instance is in transaction 3 scope.

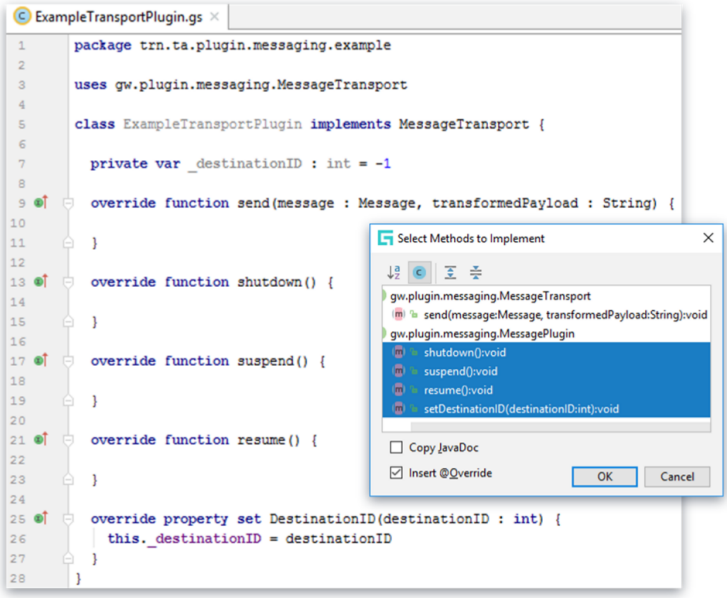
Line 8
- Create class variable.

Lines 13-14
- Method annotations added.

Line 15
- Must override send method.
- Passed the Message object and transformed payload.

Best practice:
- Pre-update rules and validation logic are **NOT** triggered from data changes made in **send** method
- Each **send** method should send the message to only one external system.
- If a message is sent to multiple systems, then there is no way to clearly track the post-send status of the message.

Implement destination plugin methods



- Implement control methods and property for messaging plugins:
  - shutdown
    - When application server shuts down
  - suspend
    - When messaging destination is suspended
  - resume
    - When messaging destination resumes
  - set DestinationID
    - When messaging plugin instance is initialized

Notes

Implement destination plugin methods

resume, suspend, and shutdown methods

- Are called by the application when the destination is resumed, suspended, or the server shuts down. Plugins should acquire or release external resources at these times. For example, a plugin may need to create or close JMS or JDBC connections, JMS Sessions, sockets, and so on.
- Typically, these methods are empty in a request plugin. In a production environment, they should be implemented for transport and reply plugins.

- The plugin should throw a Guidewire InitializationException (gw.plugin.messaging.InitializationException) from resume if the resources have not initialized successfully. (A manual resume could be attempted later and may be successful.)

set DestinationID property

- Is not related to how a message is suspended or resumed.

- It allows your destination to get its own destination ID to store it in a private variable. Your code can use the stored destination ID value for logging messages or send it to integration systems so that they can programmatically suspend/resume the destination if necessary. Keep in mind that the valid range for your destination IDs is 0 through 63, inclusive.
- Guidewire reserves all other destination IDs for built-in destinations, such as the email transport destination.

resume and suspend methods example



```
52  override function resume() {
53      // Create external web service object and set API properties
54      _bankApi = new BankVerificationAPI()
55      _bankApi.Config.Http.Authentication.Basic.Username = "externalappuser"
56      _bankApi.Config.Http.Authentication.Basic.Password = "gw"
57      _bankApi.Config.CallTimeout = 30000
58  }
59
60  override property set DestinationID(destinationID : int) {...}
63
64  override function shutdown() {...}
67
68  override function suspend() {
69      _bankApi = null
70  }
```

- To resume a destination, call the `resume` method
  - `resume` method establishes and configures the connection
- To suspend a destination, call the `suspend` method
  - `suspend` method releases any resources

Notes

resume and suspend methods example

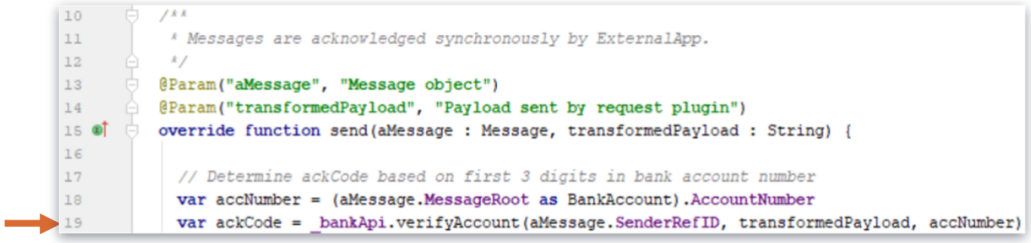
Code example
Line 54
- Create external web service object.

Lines 55-57
- Configure API authentication parameters.
- Specify CallTimeout property to prevent endlessly waiting for a server response.

Line 69
- Release external web service object.

Q: Shouldn't we add code to the shutdown method as well?
A: You can call suspend from shutdown - It doesn't really matter though as the Java garbage collector will delete the object anyway.

Send payload to web service example



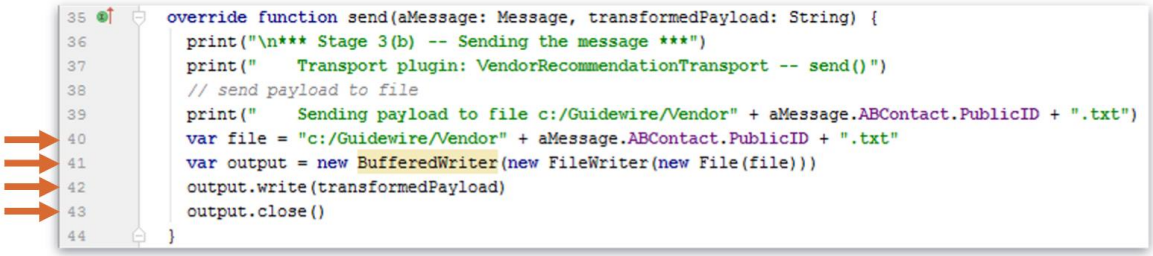
- To send a message, implement the `send` method, which the application calls with a Message entity instance argument
- The `send` method has two arguments:
  - `aMessage` is the original payload
  - `transformedPayload` is the transformed payload if that destination implemented a MessageRequest plugin

Notes

Send payload to web service example

Code example
Line 19
- Send payload to the external system.
- Messages are acknowledged synchronously by ExternalApp.

Send payload to file example



- VendorRecommendationTransport plugin
  - The send method uses Gosu's Java file I/O capabilities to write the payload to a simple flat file
  - The payload contains information about the evaluations for that vendor

Notes

Send payload to file example

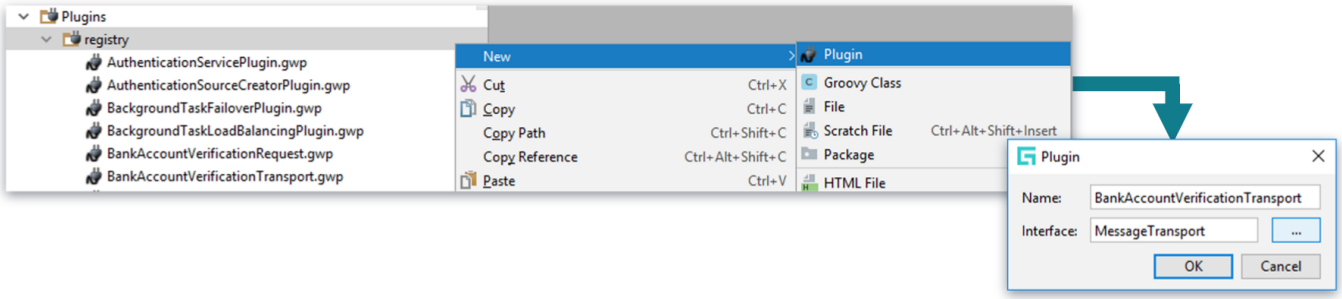
Code example
Line 40
- Text file created.

Line 41
- Creates a buffered character-output stream that uses a default-sized output buffer.

Line 42
- Writes the transformedPayload string.

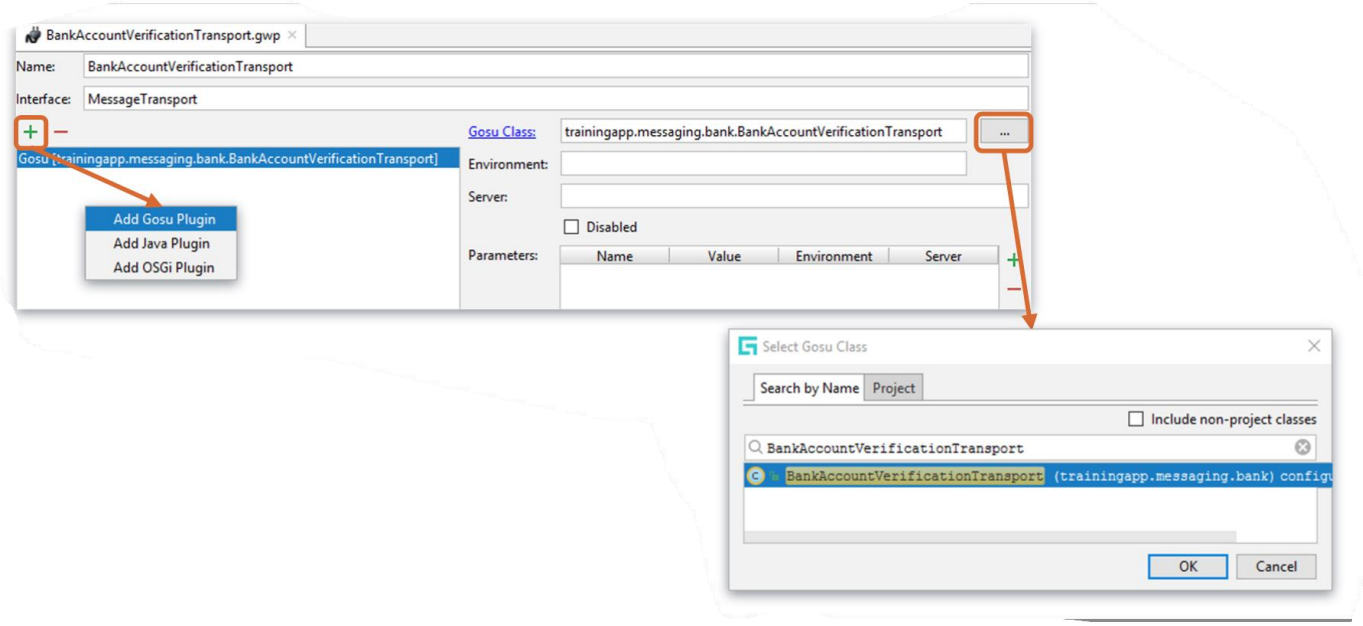
Line 43
- Closes the stream, flushing it first.

Create plugin registry



- **configuration.config.Plugins.registry**
  - Specify a new request plugin registry entry name
  - Specify the interface to be implemented
- Add the type of plugin to be implemented in the plugin name

Configure the plugin



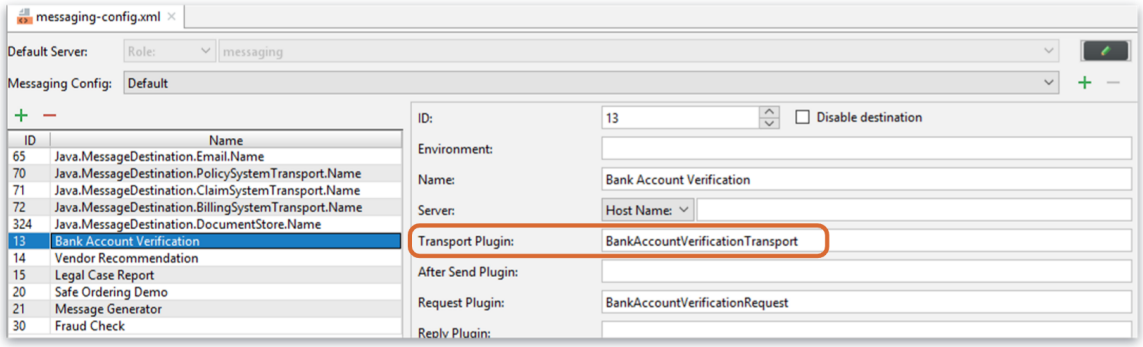
- Add Gosu Plugin
  - Optionally, select Java or OSGi plugin
- Click the ellipse (...) and select the plugin implementation class in the dialog

Notes

Configure the plugin

This plugin does not implement parameters.

Configure the destination



- Navigate to:
  - **configuration.config.Messaging.messaging-config.xml**
- The destination must point to a plugin registry element that references the appropriate Message Transport plugin

Notes

Configure the destination

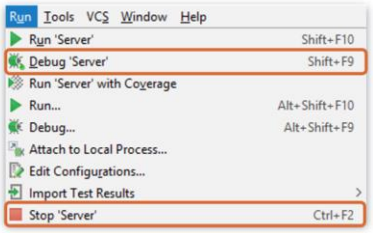
For pre-defined plugins (such as authentication), you can implement only one version of each plugin at a time. For messaging plugins, you can implement any number of MessageRequest, MessageTransport, and MessageReply plugins.

Every destination can use only one message transport plugin.

Deploy code changes

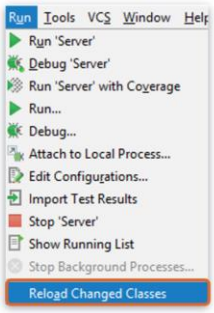
Restart Server

- New transport plugin class
- New or modified plugin registry
- New or modified destination

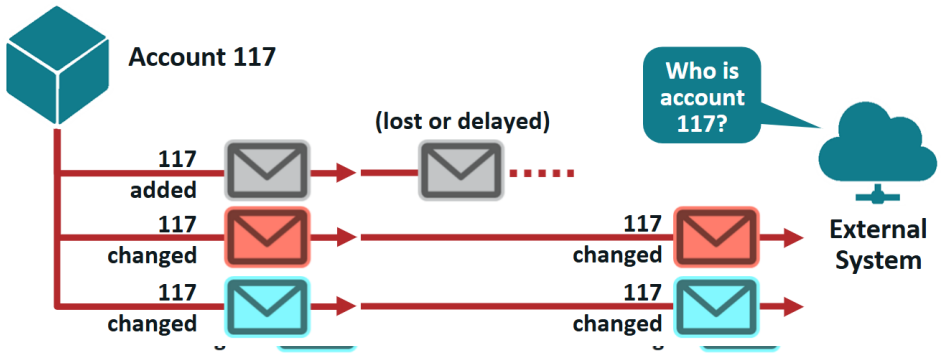


Reload Changed Classes

- Modified transport plugin class

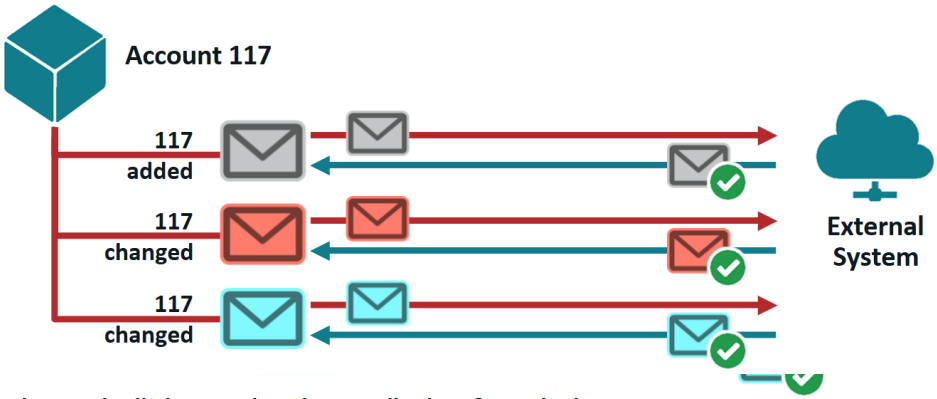


Safe ordering



- Some destinations create multiple messages for a single entity or a single set of related entities
  - Typically, they should not be sent at the same time
  - Later messages reference work done in earlier messages
- If an earlier message gets lost or arrives late, a later message will not make sense

Safe ordering



- Guidewire has a built-in mechanism called safe ordering
  - Its primary purpose is to ensure that the external system gets related messages one at a time
- When safe ordering is enforced:
  - For a given destination/entity pair, only one message is sent
  - The external system must acknowledge the first message before the next message is sent

Notes

Safe ordering

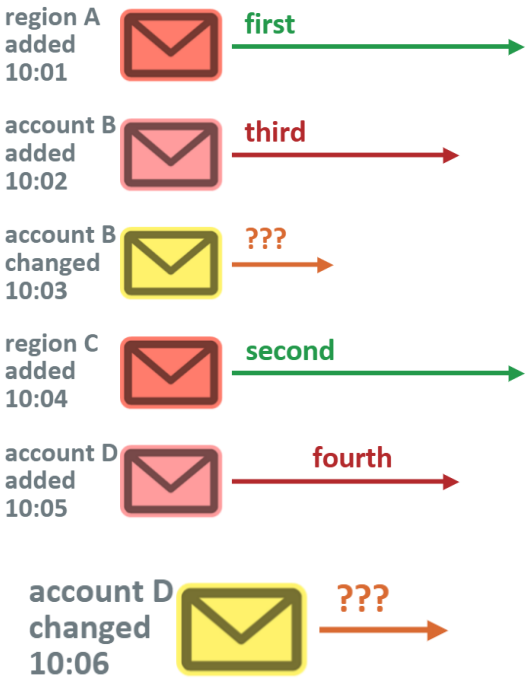
Safe ordering allows Guidewire to send messages as soon as possible, and yet prevents errors that might occur if related messages were processed out of order.

Safe ordering is not a simple first-in first-out order (FIFO). Messages are sent based on the order in which they must be received by the external system, which may or may not be identical to the order in which they were created in Event Fired rules.

This feature has large implications for messaging performance. Only one message can be in flight for each entity/destination pair. If the send queue contains 10 messages associated with different claims/accounts/contacts/producers, Guidewire can send all these safe-ordered messages immediately. However, if the send queue contains 10 safe-ordered messages for the same claim/account/contact/producer, Guidewire can send only one. Guidewire must wait for the message acknowledgement, and then at that point can send the next message, but once again can send only one.

Default ordering and sending behavior

- **Non-safe ordering messages sent…**
  - Before safe-ordering message
  - In message creation order
  - Immediately
- **Safe-ordering messages sent…**
  - After non-safe messages
  - In message creation order
  - Per entity instance, only one message at a time



Notes

Default ordering and sending behavior

The example above represents a single destination with six messages:

- Two messages apply to regions.
- Two messages apply to account B.
- Two messages apply to account D.

The time each message was created is also listed, and the messages appear in the order in which they were created.

First, Guidewire sends all the non-safe ordered messages. This is because non-safe ordered messages usually relate to administrative data that may be referenced by many other messages. Non-safe ordered messages are sent in the order they are created. (Therefore, the region A message is sent first, as it was created at 10:01. The region C message is sent second.) All non-safe ordered messages are sent immediately. Guidewire does not wait for any acknowledgement from the external system.

Next, Guidewire sends all the safe-ordered messages. As much as possible, safe-ordered messages are sent in the order that they are created. But, Guidewire sends only one message for a given entity/destination pair at a time. So, the send time of later messages for a given entity/destination pair depends upon when acknowledgement is received for earlier messages in that entity/destination pair. (Therefore, the **account B added** message is sent third. The **account B changed** message cannot be sent until the **account B added** message is acknowledged.

Assuming there is no immediate acknowledgement of the message, the **account D added** message is sent fourth. The order and timing of the **account B changed** and **account D changed** messages is unknown because the sending of these messages depends upon the acknowledgement of the corresponding account added messages.)

Strict mode

Destinations have an option known as "strict mode". By default, it is turned off. Checking the destination's "Strict mode" turns it on.

When a destination is in strict mode:

- Non-safe-ordered messages are sent one at a time. The next non-safe ordered message is not sent until the previous non-safe ordered message is acknowledged.
- Safe-ordered messages are not sent until all non-safe-ordered messages have been sent and acknowledged.

This option is intended solely for destinations that send messages for multiple types of entities, and the order in which those messages are sent affects the external system's ability to understand them. Strict mode should be used cautiously, as it potentially affects the through-put of the destination because a single non-safe-ordered message that does not receive a positive acknowledgement will prevent the destination from sending further messages.

InsuranceSuite safe ordering

ClaimCenter

- Safe ordering applies only to messages related directly or indirectly to instances of **Claim**
- Alternative Primary Entity can be configured in a destination to safe order for the **Contact** entity

PolicyCenter

- Safe ordering applies only to messages related directly or indirectly to instances of **Account**
- Alternative Primary Entity can be configured in a destination to safe order for the **Contact** entity

BillingCenter

- Safe ordering is not used for any primary entity messages
- For any destination, you can enable safe ordering for one of the supported primary entity types:
  - **Account**
  - **Contact**
  - **PolicyPeriod**
  - **Producer**



Notes

InsuranceSuite safe ordering

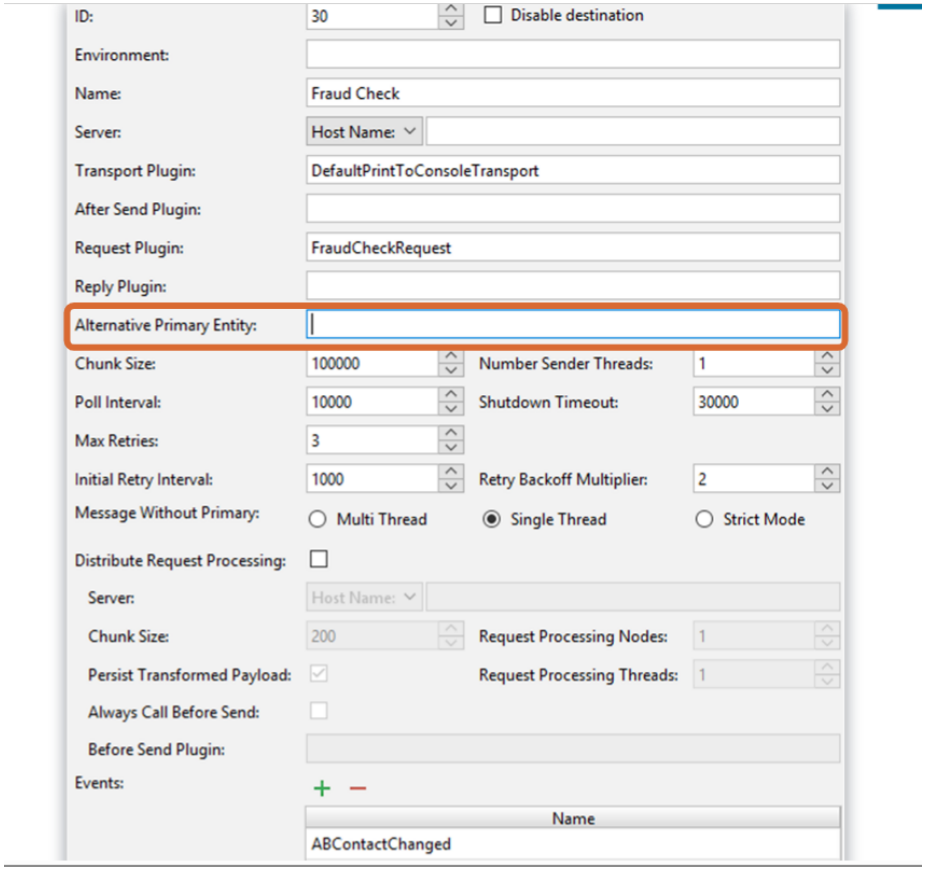
For most destinations, the default safe ordering behavior of the primary entity (CC-Claim, PC-Account) is appropriate. However, some destinations may represent integration points to a contact management system such as Guidewire ContactManager. For these integration points, each message may pertain to only one contact, and there are no issues around sending multiple messages for the same claim/account at the same time provided that each message pertains to a different contact. For these destinations, you can define that safe ordering should be enforced at the Contact level. This means a given claim may have multiple messages in flight, but it will never have more than one message in flight for any given contact .

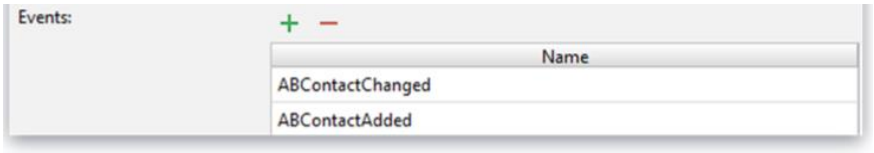
ClaimCenter and PolicyCenter
To configure a destination to do safe ordering at the Contact level, select **Contact** in the Alternative Primary Entity field. Contact is the only alternative primary entity in ClaimCenter and PolicyCenter.

BillingCenter
Unlike ClaimCenter (where most business information is owned uniquely by a claim) or PolicyCenter (where most business information is owned uniquely by an account), BillingCenter does not have a single entity whose instances uniquely own most of the business information. Therefore, BillingCenter does not have a single entity that is the default safe-ordered entity. The default is not to use safe ordering, and safe ordering must be enabled for each destination with the appropriate entity. Because safe ordering reduces the likelihood of errors on the external system and can increase performance, the best practice is to enable safe ordering for every destination that can make use of it. To configure a destination to do safe ordering, select the appropriate entity type (Account, Contact, PolicyPeriod, or Producer) in the Alternative Primary Entity field.

ContactManager safe ordering

- In ContactManager (and TrainingApp), messages are safe ordered based on ABContact
  - This is true for every destination
  - There are no alternative primary entity types





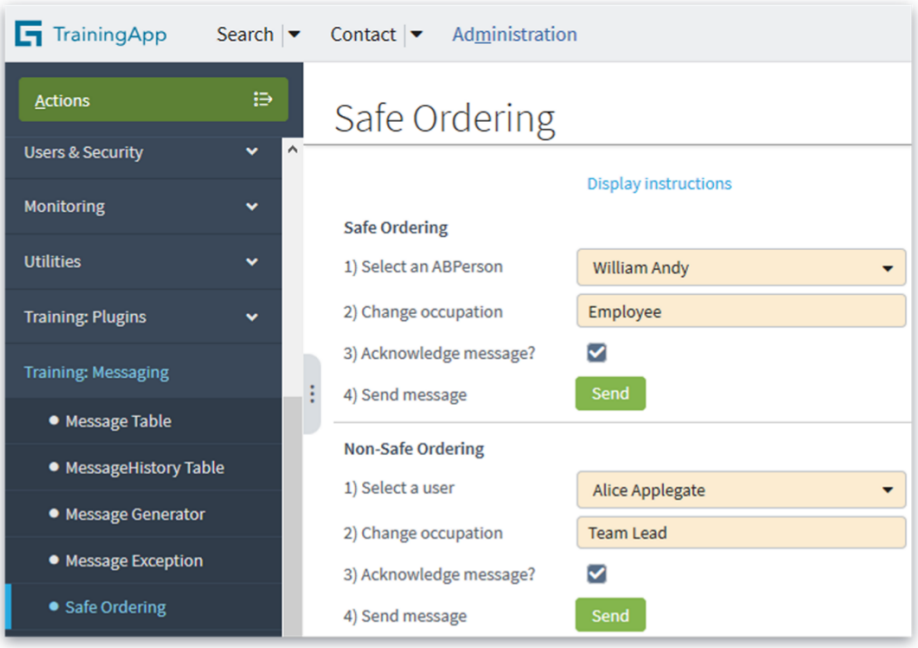
Notes

ContactManager safe ordering

Safe ordering is a messaging feature that causes related messages to send in the same order they were sent. This is called safe order. For each messaging destination, the application groups messages by their associated primary object. For important definitions relevant to this topic, see **Primary Entity and Primary Object** in the product documentation.

Safe ordering demo in TrainingApp

Administration → Training: Messaging → Safe Ordering



Notes

Safe ordering demo in TrainingApp

TrainingApp includes a safe ordering demo. Using this demo, you can select and change either an ABContact or a User. You can send a message with acknowledgement or without acknowledgement. Message activity is printed to the console. Using this demo, you can see that:

- If two messages are sent for the same ABContact (a safe-ordering entity), the second message is sent only if the first message is acknowledged.
- If two messages are sent for the same User (a non-safe-ordering entity), the second message is always sent, regardless of the acknowledgement of the first message.

The demo is located on the Administration tab's **Training: Messaging - Safe Ordering** screen. Detailed instructions for how to use the demo are also included. To view the instructions, click the **Display instructions** link at the top of the detail view.

Message send failure

Transport or Request plugin error



- When Transport or Request plugins throws an exception, the Guidewire application automatically:
  - Compares message send attempts to Max Retries
  - Waits the Initial Retry Interval
  - Retries to send message
  - Increases the time between resends (Retry Backoff Multiplier)
  - Continues until reaching Max Retries
- Parameters defined in message destination

Notes

Transport or Request plugin error

An exception is handled differently than a reported error for a message. There are various types of exceptions such as a java.net.ConnectionException or gw.xml.ws.WsiAuthenticationException.

Sometimes a message transport plugin has a send error and expects it to be temporary, for example, the external system is not available. To support this common use case, if the message transport plugin throws an exception from its send method, the Guidewire application will retry to send the message after a specified amount of delay time, and will continue to retry sending the message multiple times.

When an exception is thrown in the transport plugin send method, the Guidewire application automatic response is to retry sending the message until reaching the maximum number of retries. The message's destination configuration defines the value for the maximum number of retries. When reaching the maximum number of retries, Guidewire suspends the destination.

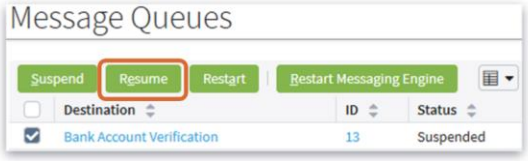
Destinations have specific parameters that impact how messages are retried.

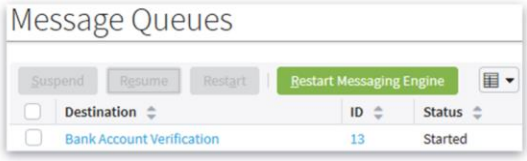
- **Initial Retry Interval** – The amount of time in milliseconds after a retryable error to retry a sending a message.
- **Max Retries** – The number of automatic retries to attempt before suspending the messaging destination.
- **Retry Backoff Multiplier** – The amount to increase the time between retries, specified as a multiplier of the time previously attempted. For example, if the last retry time attempted was 5 minutes, and the retry backoff multiplier is set to 2, the application attempts the next retry in 10 minutes.
- **Shutdown Timeout** – Messaging plugins have methods to handle the administrative commands suspend, resume, and preparing for the messaging system to shutdown. The shutdown timeout value is the length of time to wait before attempting to shutdown the messaging subsystem.

Error administration



- Guidewire automatically suspends a destination when exceeding Max Retries
  - No further messages are sent for destination
- An administrator must manually resume the destination
  - The message that caused the suspend will be resent, up to the maximum retries if necessary





Notes

Error administration

A message causes the destination to be suspended when:

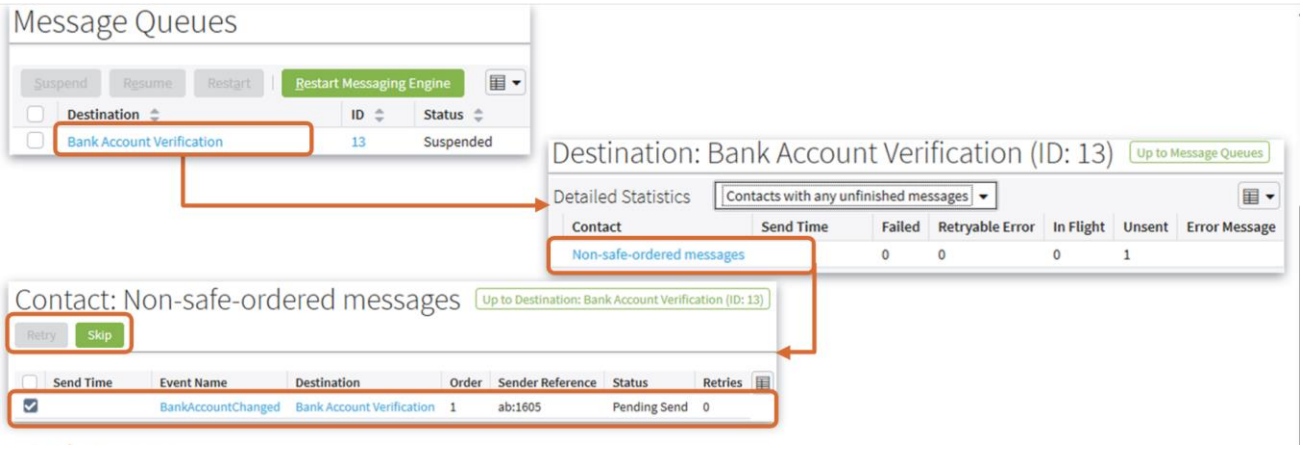
- The beforeSend or send method throws an exception, AND
- The message has reached the maximum retry limit

In order for an additional messages to be processed for the messaging destination, you need to skip any messages reporting an error and to resume the suspended destination. An administrator can manually skip a message. It is also possible for custom code to skip messages, for example, from within a batch process. If a message is safe ordered, the next message for the destination/entity pair will not be sent until the message is either resent and successfully acknowledged or skipped.

Messaging plugins have methods to suspend, resume, and shutdown the system. During the suspend, shutdown, and resume methods of the plugin, the plugin must not call any APIs that suspend or resume messaging destinations. This includes—but is not limited to—MessageToolsAPI web service APIs. Doing so creates circular application logic.

Retry and skip messages





Notes

Retry and skip messages

Every Guidewire application has an Message Queues screen on the Administration tab. It is used to administer messages.
To manually retry or skip a specific message:

- From the Message Queues screen, click the Destination name.
- From the Destination: <Destination Name> screen, click the appropriate link.
  - If the destination has messages associated to non-safe-ordered entities, the screen will have a Non-safe-ordered messages link.
  - If the destination has messages associated to safe-ordered entities, the screen will have a Safe-ordered messages link for each safe-ordered entity.
- From the next screen, click the appropriate message or messages. Then, click Retry or Skip.
  - To retry, the destination must have a status of Started.

When you manually retry a message, Guidewire attempts to resend the message by calling the same methods as the first attempt: the beforeSend method (if a reply plugin has been implemented) and the send method. If the issue that caused the original problem has not been rectified, the message will cause another exception, and after the Max Retries has been exceeded, the destination will be suspended again.

When you skip a message, the message is moved from the Message table to the MessageHistory table. The status of the message is set to "13" (skipped). Then the next message in its order (if there is one) can be sent. Note that skipping a message implies that the information that it was expected to carry to the external system is lost. When programming for messaging, care should be taken to avoid programming in such a way that large numbers of messages may be inadvertently skipped.

Contrasting retry scenarios

	Initial send error (Automatic retry during send)	External system error (Automatic retry after send)	Manual retry
What is the problem?	Exception thrown in **beforeSend** or **send** methods	* Discussed in Message Acknowledgment	Message needs manual intervention
What executes the retry?	Internal Guidewire behavior	*	Administrator using Retry button
Should each attempt use the same SenderRefID	**Yes**	*	Yes
What specifies the max retry number?	Max Retries property for messaging destination	*	n/a
What happens when the max number of retires is reached?	The destination is suspended	*	n/a

Contrasting retry scenarios

A message could be retried automatically during the sending of the message.
During the sending of a message, a message has a status of "(1) Pending send". Guidewire automatically retries sending of a message when an exception is thrown in the beforeSend or send method. The message destination's Max Retries property determines the maximum number of retries. Because the message has not yet been sent, each attempt should use the same SenderRefID because it is still the same message – each send attempt does not create a new message. The message request code should set the SenderRefID only if it is null. If a given message exceeds the maximum number of retries, the Guidewire application suspends the destination. The Guidewire application will not send any more messages for a suspended destination. You must resume the destination to remove the suspension.

Note: If a message is sent successfully, but the external system replies with an error, then a ***NEW*** message is sent. The new message is a copy of the old message that generated the error at the external system. Since this is a ***NEW*** message, a new SenderRefID will be generated.

A message could be retried manually.
This occurs when a message requires manual intervention. This could be necessary for several reasons, such as a message that caused an exception during send where the situation has now been rectified, or a message whose payload needed to be modified manually, or a message for which the external system never provided a response. An administrator retries the message manually by selecting the message from one of the Message Administration screens and clicking the Retry button. There is no notion of maximum retries for a message retried manually.

Note: A message could be retried automatically after the sending of the message.

Answer

Every destination can optionally have a request plugin, but it must have a transport plugin. Why is the transport plugin required?

The transport plugin is required because it is used to send the message when it is processed from the Message table. Without a transport plugin, messages for a given destination would have no way of being sent to the corresponding external system.

Answer

Can one destination have two transport plugins? Can one transport plugin be used by two destinations?

A destination can't have two transport plugins, but a transport plugin class can be used by two (or more) similar destinations.

Why should the send method never send a message to multiple external systems?

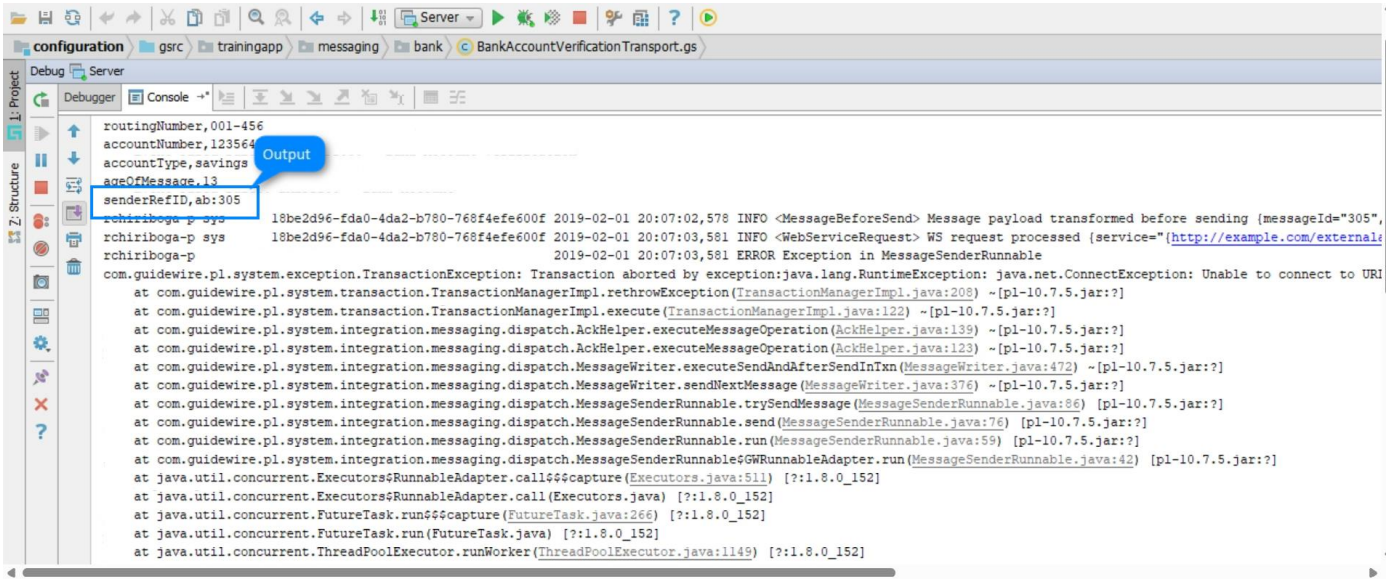
The message acknowledgement architecture assumes that each message is sent to only one destination. If a given send method sent a message to multiple destinations, then there could be multiple acknowledgements, errors, or unprocessed messages, and the messaging architecture would not be able to identify the state of the message.

Demonstration

BankAccountVerificationTransport.gs

```
1  package trainingapp.messaging.bank
2
3  uses gw.plugin.messaging.MessageTransport
4  uses trainingapp.webservice.bank.bankaccountverificationwsc.bankverificationapi.BankVerificationAPI
5
6  class BankAccountVerificationTransport implements MessageTransport {
7
8    /**
9     * Messages are acknowledged synchronously by ExternalApp.
10    */
11   @Param("aMessage", "Message object")
12   @Param("transformedPayload", "Payload sent by request plugin")
13   override function send(aMessage : Message, transformedPayload : String) {
14     // Create external web service object and set API properties
15     var bankAPI = new BankVerificationAPI()
16     bankAPI.Config.Http.Authentication.Basic.Username = "externalappuser"
17     bankAPI.Config.Http.Authentication.Basic.Password = "gw"
18     bankAPI.Config.CallTimeout = 30000
19     // Determine ackCode based on first 3 digits in bank account number
20     var accNumber = (aMessage.MessageRoot as BankAccount).AccountNumber
21     var ackCode = bankAPI.verifyAccount(aMessage.SenderRefID, transformedPayload, accNumber)
22     // Synchronous acknowledgment
23     switch (ackCode) {
24       case "000":
25         // Message acknowledged - Invalid
26         aMessage.reportAck()
27         (aMessage.MessageRoot as BankAccount).IsVerified = VerificationStatus.TC_INVALID
28         break
29       case "999":
```

```
*** Stage 3 -- Optionally transform the message payload ***
    Request plugin: BankAccountVerificationRequest -- beforeSend()
    Sending transformed payload to console:
contact,ab:10
bankName,Succeed Credit Union
routingNumber,001-456
accountNumber,1235647890
accountType,savings
ageOfMessage,3
senderRefID,ab:305
rchiriboga-p sys      18be2d96-fda0-4da2-b780-768f4efe600f 2019-02-01 20:06:52,501 INFO <MessageBeforeSend> Message payload transformed before sending {messageId="305",
rchiriboga-p sys      18be2d96-fda0-4da2-b780-768f4efe600f 2019-02-01 20:06:53,507 INFO <WebServiceRequest> WS request processed {service="{http://example.com/external
rchiriboga-p                                       2019-02-01 20:06:53,508 ERROR Exception in MessageSenderRunnable
com.guidewire.pl.system.exception.TransactionException: Transaction aborted by exception:java.lang.RuntimeException: java.net.ConnectException: Unable to connect to URI
    at com.guidewire.pl.system.transaction.TransactionManagerImpl.rethrowException(TransactionManagerImpl.java:208) [pl-10.7.5.jar:?]
    at com.guidewire.pl.system.transaction.TransactionManagerImpl.execute(TransactionManagerImpl.java:122) ~[pl-10.7.5.jar:?]
    at com.guidewire.pl.system.integration.messaging.dispatch.AckHelper.executeMessageOperation(AckHelper.java:139) ~[pl-10.7.5.jar:?]
    at com.guidewire.pl.system.integration.messaging.dispatch.AckHelper.executeMessageOperation(AckHelper.java:123) ~[pl-10.7.5.jar:?]
    at com.guidewire.pl.system.integration.messaging.dispatch.MessageWriter.executeSendAndAfterSendInTxn(MessageWriter.java:472) ~[pl-10.7.5.jar:?]
    at com.guidewire.pl.system.integration.messaging.dispatch.MessageWriter.sendNextMessage(MessageWriter.java:376) ~[pl-10.7.5.jar:?]
    at com.guidewire.pl.system.integration.messaging.dispatch.MessageSenderRunnable.trySendMessage(MessageSenderRunnable.java:86) [pl-10.7.5.jar:?]
```


Server
configuration
gsrc
trainingapp
messaging
bank
BankAccountVerificationTransport.gs
Debug
Server
Debugger
Console
Output
routingNumber,001-456
accountNumber,123564
accountType,savings
ageOfMessage,13
senderRefID,ab:305
rchiriboga-p sys 18be2d96-fda0-4da2-b780-768f4efe600f 2019-02-01 20:07:02,578 INFO <MessageBeforeSend> Message payload transformed before sending {messageId="305",
rchiriboga-p sys 18be2d96-fda0-4da2-b780-768f4efe600f 2019-02-01 20:07:03,581 INFO <WebServiceRequest> WS request processed {service="{http://example.com/external
rchiriboga-p 2019-02-01 20:07:03,581 ERROR Exception in MessageSenderRunnable
com.guidewire.pl.system.exception.TransactionException: Transaction aborted by exception:java.lang.RuntimeException: java.net.ConnectException: Unable to connect to URI
at com.guidewire.pl.system.transaction.TransactionManagerImpl.rethrowException(TransactionManagerImpl.java:208) ~[pl-10.7.5.jar:?]
at com.guidewire.pl.system.transaction.TransactionManagerImpl.execute(TransactionManagerImpl.java:122) ~[pl-10.7.5.jar:?]
at com.guidewire.pl.system.integration.messaging.dispatch.AckHelper.executeMessageOperation(AckHelper.java:139) ~[pl-10.7.5.jar:?]
at com.guidewire.pl.system.integration.messaging.dispatch.AckHelper.executeMessageOperation(AckHelper.java:123) ~[pl-10.7.5.jar:?]
at com.guidewire.pl.system.integration.messaging.dispatch.MessageWriter.executeSendAndAfterSendInTxn(MessageWriter.java:472) ~[pl-10.7.5.jar:?]
at com.guidewire.pl.system.integration.messaging.dispatch.MessageWriter.sendNextMessage(MessageWriter.java:376) ~[pl-10.7.5.jar:?]
at com.guidewire.pl.system.integration.messaging.dispatch.MessageSenderRunnable.trySendMessage(MessageSenderRunnable.java:86) [pl-10.7.5.jar:?]
at com.guidewire.pl.system.integration.messaging.dispatch.MessageSenderRunnable.send(MessageSenderRunnable.java:76) [pl-10.7.5.jar:?]
at com.guidewire.pl.system.integration.messaging.dispatch.MessageSenderRunnable.run(MessageSenderRunnable.java:59) [pl-10.7.5.jar:?]
at com.guidewire.pl.system.integration.messaging.dispatch.MessageSenderRunnable$GWRunnableAdapter.run(MessageSenderRunnable.java:42) [pl-10.7.5.jar:?]
at java.util.concurrent.Executors$RunnableAdapter.call$$$capture(Executors.java:511) [?:1.8.0_152]
at java.util.concurrent.Executors$RunnableAdapter.call(Executors.java) [?:1.8.0_152]
at java.util.concurrent.FutureTask.run$$$capture(FutureTask.java:266) [?:1.8.0_152]
at java.util.concurrent.FutureTask.run(FutureTask.java) [?:1.8.0_152]
at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149) [?:1.8.0_152]